

8-2 Space Flight Planning Problem

8-2 太空飞行计划问题

Solving Maximum Weight Closure using Max-Flow Min-Cut

使用最大流最小割解决最大权闭合子图问题



The Goal: Maximize Net Profit

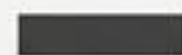
目标：最大化净收益

Professor W wants to plan a space flight. The goal is simple: make the most money.

W 教授想要计划一次太空飞行。
目标很简单：赚最多的钱。



Total Revenue
from Experiments
实验总收入



Total Cost of
Instruments
仪器的总费用



Net Profit
净收益

**Net Profit = (Total Revenue from Experiments)
- (Total Cost of Instruments)**

净收益 = (实验总收入) - (仪器的总费用)

The Rules of the Mission

任务的规则

There are clear rules for planning the flight.
计划飞行有明确的规则。



- **Experiments Earn Money:** Each experiment (E_j) provides revenue (p_j).
实验带来收入：每个实验 (E_j) 带来收入 (p_j)。



- **Instruments Cost Money:** Each instrument (I_k) has a cost (c_k).
仪器需要花费：每个仪器 (I_k) 需要花费 (c_k)。



- **The Key Constraint:** If you choose an experiment, you **MUST** purchase all of its required instruments.
关键约束：如果选择了一个实验，就 **必须** 购买它所需要的所有仪器。

A Concrete Example

一个具体的例子

Let's look at the data from our problem.
让我们看一下我们问题中的数据。

Item	Details
Experiment 1	Earns 10 , Needs Instruments {1, 2}
Experiment 2	Earns 25 , Needs Instruments {2, 3}
Instrument 1 Cost	5
Instrument 2 Cost	6
Instrument 3 Cost	7

The Challenge: It's a Selection Problem

挑战：这是一个选择问题

We can't just pick the most profitable experiment (Experiment 2, earns 25).
我们不能只选择利润最高的实验（实验2，收入25）。

Why? Because its cost ($\text{Inst 2} + \text{Inst 3} = 6 + 7 = 13$) might not be optimal. Maybe choosing both experiments is better, even with the higher total instrument cost.
为什么？因为它的成本（仪器2 + 仪器3 = $6 + 7 = 13$ ）可能不是最优的。也许选择两个实验更好，即使仪器总成本更高。

This is a dependency problem. We need a systematic way to make the best choice.
这是一个依赖问题。我们需要一个系统性的方法来做出最佳选择。

The Theoretical Model 理论模型

This type of challenge is a classic computer science problem.
这类挑战是一个经典的计算机科学问题。

Problem Name: Maximum Weight Closure Problem
问题名称: 最大权闭合子图问题

Solution Method: We transform the problem into a **Minimum Cut** problem on a special graph.
解决方法: 我们将问题转化为一个特殊图上的 **最小割** 问题。



Profit Problem
利润问题

Transformation
建模 (Modeling)



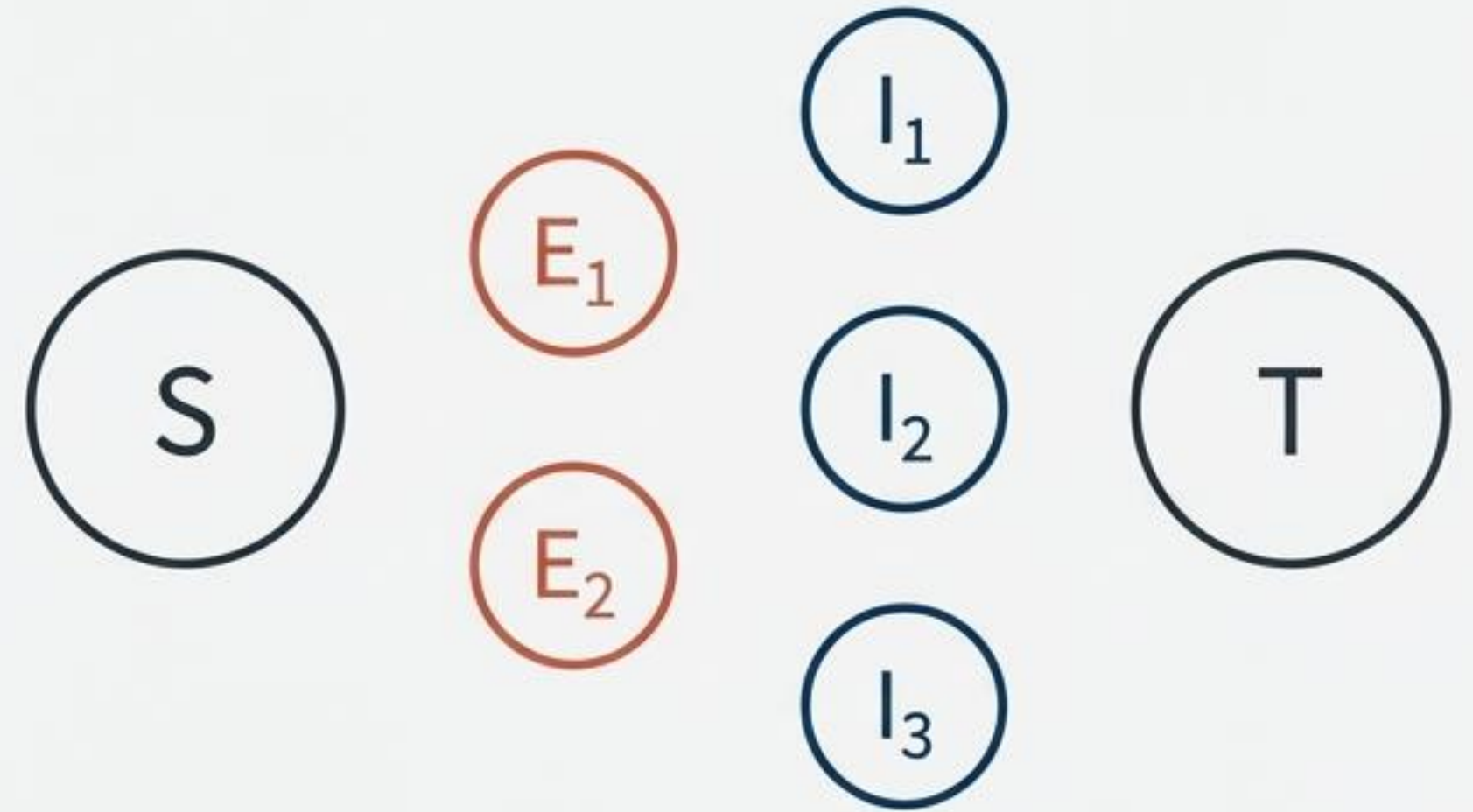
Min-Cut Graph Problem
最小割图问题

Step 1: Building the Graph - The Nodes

步骤 1: 构建图 - 节点

We will build a flow network. First, we need nodes for every part of our problem.
我们将建立一个流网络。首先，我们需要为问题的每个部分创建节点。

1. A Source node (S) - The start of all flow.
一个源点 (S) - 所有流的起点。
2. A Sink node (T) - The end of all flow.
一个汇点 (T) - 所有流的终点。
3. Experiment Nodes ($E_1, E_2, \dots$)
实验节点 ($E_1, E_2, \dots$)
4. Instrument Nodes ($I_1, I_2, \dots$)
仪器节点 ($I_1, I_2, \dots$)

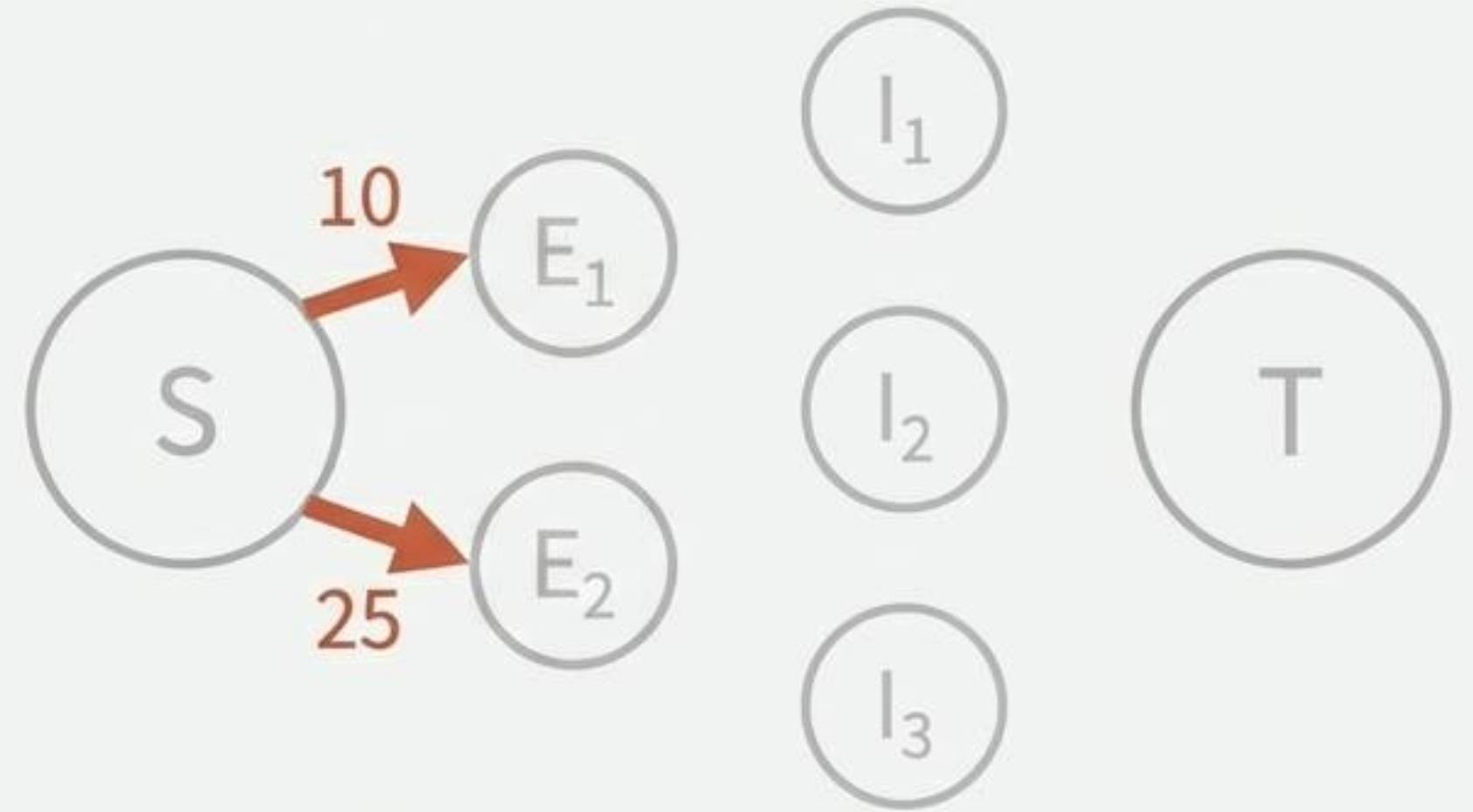


Step 2: Adding Edges - Revenue

步骤 2: 添加边 - 收入

We represent potential revenue as flow from the Source.
我们用从源点流出的流量来代表潜在收入。

- **Rule:** Connect Source (S) to every Experiment (E_j).
规则：将源点 (S) 连接到每个实验 (E_j)。
- **Capacity:** The edge capacity is the experiment's revenue (p_j).
容量：边的容量是实验的收入 (p_j) 。



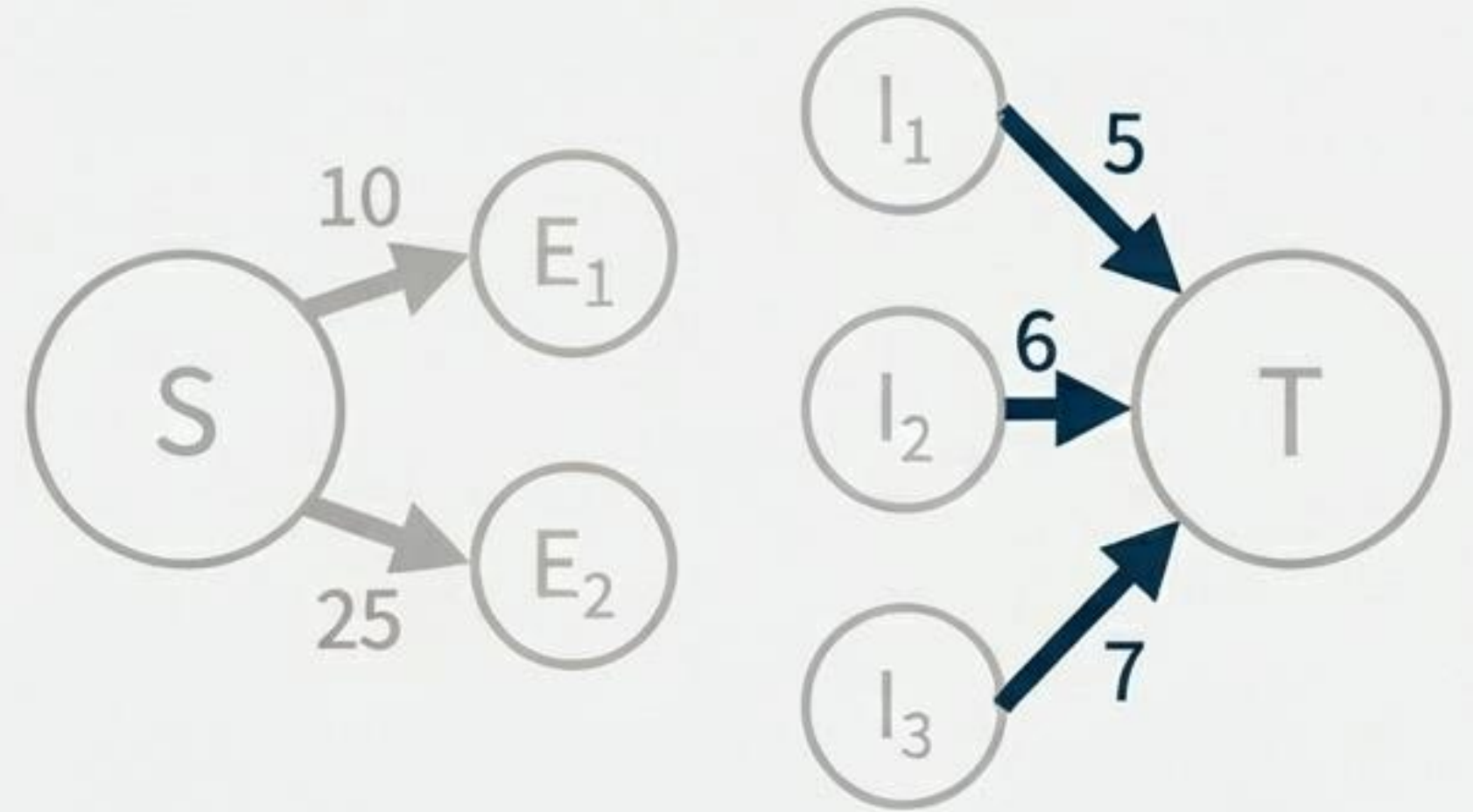
Step 3: Adding Edges - Costs

步骤 3: 添加边 - 成本

We represent instrument costs as flow to the Sink.

我们用流向汇点的流量来代表仪器成本。

- **Rule:** Connect every Instrument (I_k) to the Sink (T).
规则：将每个仪器 (I_k) 连接到汇点 (T)。
- **Capacity:** The edge capacity is the instrument's cost (c_k).
容量：边的容量是仪器的费用 (c_k) 。

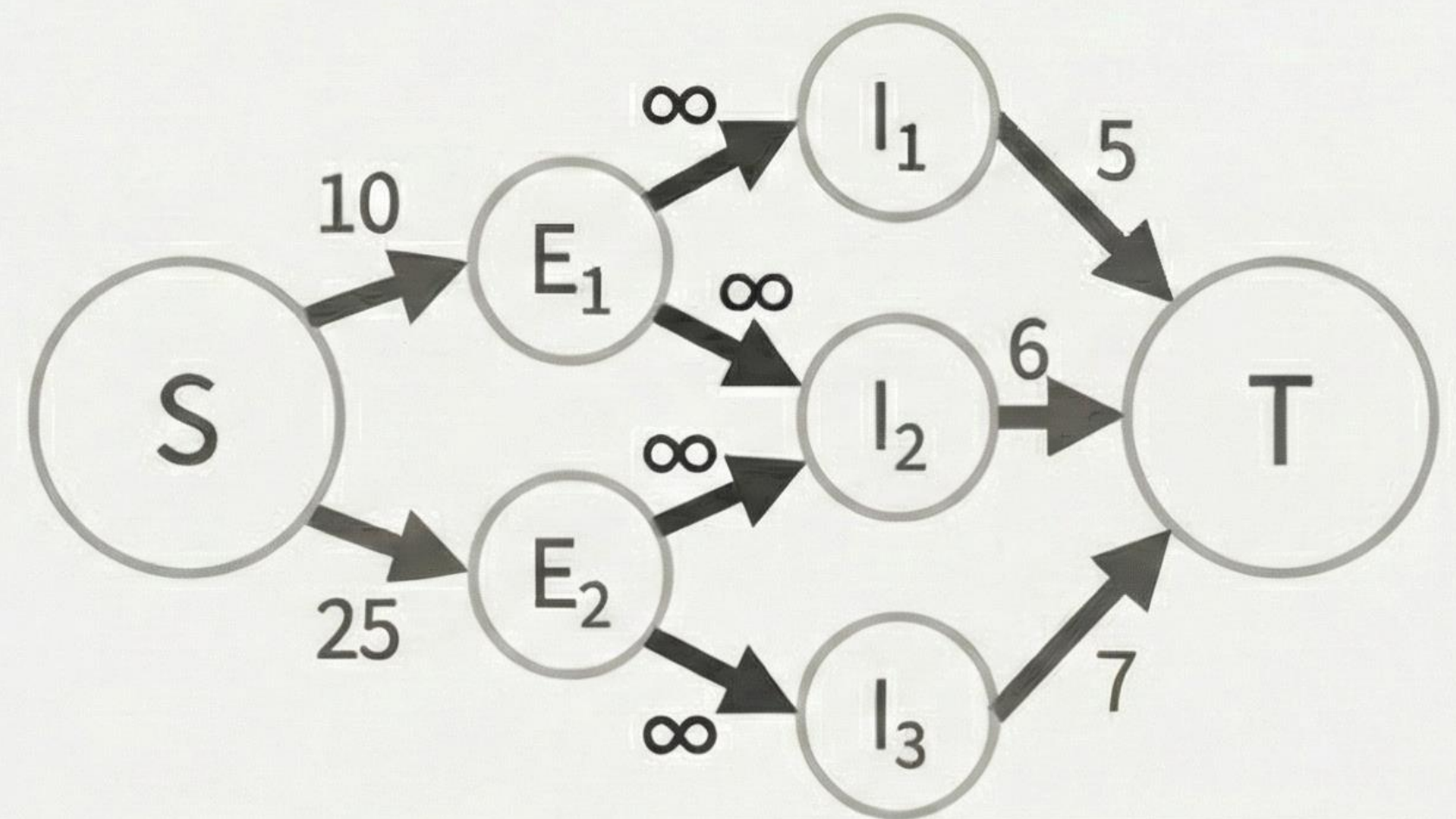


Step 4: Adding Edges - Dependencies

步骤 4: 添加边 - 依赖关系

This is the most important step. We must enforce the constraint.
这是最重要的一步。我们必须强制执行约束。

- **Rule:** Connect an Experiment (E_j) to every Instrument (I_k) it needs.
规则：将一个实验 (E_j) 连接到它所需的每个仪器 (I_k)。
- **Capacity:** Infinity (∞).
容量：无穷大 (∞)。



Why Infinity? 为什么是无穷大?

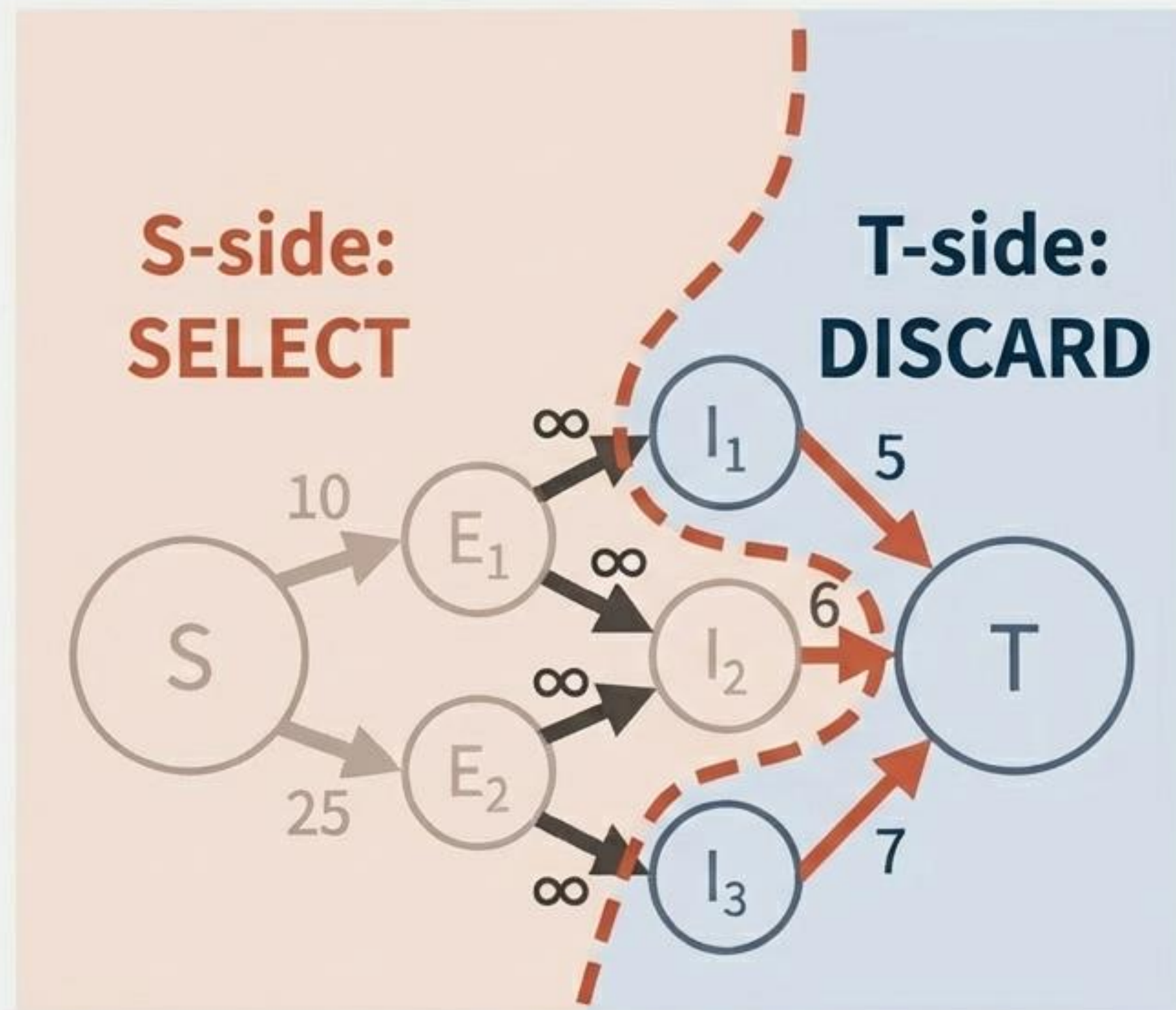
An edge with infinite capacity can **never be cut** by a minimum-cut algorithm. This forces the logic: if you select an experiment, you must also select its instruments.
最小割算法永远不会切断容量无穷大的边。这强制了我们的逻辑：如果选择一个实验，就必须同时选择其所需的仪器。

What is a “Min-Cut”?

什么是“最小割”？

A ‘cut’ is a partition that separates all nodes into two groups: an S-side and a T-side.

“割”是一个划分，它将所有节点分成两组：S侧和T侧。



The algorithm finds the cut where the sum of capacities of edges crossing from the S-side to the T-side is minimized. This sum represents our **total loss**.

算法会找到一条割，使得从 S 侧到 T 侧的边的总容量最小。这个总容量就代表我们的 **总损失**。

The Magic Formula

神奇的公式

The Max-Flow Min-Cut theorem connects everything.
最大流最小割定理将一切联系起来。

Min Cut = Minimum Loss
最小割 = 最小损失

To get the maximum profit, we subtract the minimum possible loss from the maximum possible revenue.
为了获得最大利润，我们从所有可能的总收入中减去最小的可能损失。

Max Profit = (Total Revenue of ALL Experiments) - Min Cut
最大利润 = (所有实验的总收入) - 最小割

Solution in Code: Edmonds-Karp

代码实现：Edmonds-Karp 算法

We will implement this using the **Edmonds-Karp Algorithm**, a classic method for finding Max-Flow.
我们将使用 **Edmonds-Karp 算法** 来实现，这是一种经典的计算最大流的方法。

Our Python Code Structure:

我们的 Python 代码结构：



1. A ``MaxFlowSolver`` class to hold the graph and logic.
一个 ``MaxFlowSolver`` 类来存储图和算法逻辑。



2. A ``bfs`` function to find a path with available capacity.
一个 ``bfs`` 函数来寻找有可用容量的路径。



3. A ``max_flow`` function to calculate the total flow (our min-cut value).
一个 ``max_flow`` 函数来计算总流量（即我们的最小割值）。



4. The main script to parse input and build the graph.
主程序，用于解析输入并构建图。

```
import sys
from collections import deque

class MaxFlowSolver:
    def __init__(self, num_nodes, source, sink):
        self.num_nodes = num_nodes
        self.source = source
        self.sink = sink
        # Graph stores the capacity and current flow of edges
        # 图存储边的容量和当前流量
        # capacity[u][v] stores total capacity from u to v
        # capacity[u][v] 存储从 u 到 v 的总容量
        self.capacity = [[0] * num_nodes for _ in range(num_nodes)]
        self.graph = [[] for _ in range(num_nodes)]

    def add_edge(self, u, v, cap):
        # Add forward edge
        # 添加正向边
        self.graph[u].append(v)
        self.capacity[u][v] = cap
        # Add backward edge (for residual graph flow) with 0 capacity initially
        # 添加反向边 (用于残量网络流), 初始容量为 0
        self.graph[v].append(u)
        # Note: self.capacity[v][u] remains 0 initially
        # 注意: self.capacity[v][u] 初始保持为 0

    def bfs(self, parent):
        # Find an augmenting path from source to sink using BFS
        # 使用 BFS (广度优先搜索) 查找从源点到汇点的增广路径
        visited = [False] * self.num_nodes
        queue = deque([self.source])
        visited[self.source] = True
        parent[self.source] = -1

        while queue:
            u = queue.popleft()
            for v in self.graph[u]:
                # If v is not visited and there is residual capacity
                # 如果 v 未被访问且存在残量容量 (即可以有流量通过)
                if not visited[v] and self.capacity[u][v] > 0:
                    queue.append(v)
                    visited[v] = True
                    parent[v] = u
                    if v == self.sink:
                        return True

        return False

    def max_flow(self):
        max_flow_value = 0
        parent = [-1] * self.num_nodes

        # While there is a path from source to sink in residual graph
        # 当残量图中存在从源点到汇点的路径时
        while self.bfs(parent):
            path_flow = float('Inf')
            s = self.sink

            # Find the minimum residual capacity in the path found
            # 找出所发现路径中的最小残量容量 (即路径的瓶颈)
            while s != self.source:
                path_flow = min(path_flow, self.capacity[parent[s]][s])
                s = parent[s]

            # Update residual capacities of the edges and reverse edges
            # 更新边和反向边的残量容量
            max_flow_value += path_flow
            v = self.sink
            while v != self.source:
                u = parent[v]
                self.capacity[u][v] -= path_flow # Decrease forward capacity
                # 减少正向边的容量
                self.capacity[v][u] += path_flow # Increase backward capacity
                # 增加反向边的容量
                v = parent[v]

        return max_flow_value

    def get_selected_set(self):
        # Perform BFS/DFS one last time from Source on the residual graph
        # to see which nodes are reachable.
        # 在残量网络上从源点最后执行一次 BFS/DFS, 查看哪些节点是可达的。
        # (These reachable nodes represent the "Selected" set in the Min-Cut / S-Side)
        # (这些可达节点代表最小割中被“选中”的集合 / S侧)
        visited = [False] * self.num_nodes
        queue = deque([self.source])
        visited[self.source] = True

        while queue:
            u = queue.popleft()
            for v in self.graph[u]:
                if not visited[v] and self.capacity[u][v] > 0:
                    visited[v] = True
                    queue.append(v)

        return visited
```

****Code: Step 1 - Initialization****

代码：步骤 1 - 初始化

We start with a class to store our graph's state.

我们从一个类开始，用来存储图的状态。

```
from collections import deque

class MaxFlowSolver:
    def __init__(self, num_nodes, source, sink):
        self.num_nodes = num_nodes
        self.source = source
        self.sink = sink

        # self.capacity[u][v] stores the capacity from u to v
        # self.capacity[u][v] 存储从 u 到 v 的容量
        self.capacity = [[0] * num_nodes for _ in range(num_nodes)]

        # Adjacency list for graph traversal
        # 邻接表，用于图的遍历
        self.graph = [[] for _ in range(num_nodes)]
```

****Code: Step 2 - Adding Edges****

代码：步骤 2 - 添加边

When we add an edge, we also add a 'backward' edge for the residual graph.

当我们添加一条边时，我们也会为残余网络添加一条“反向”边。

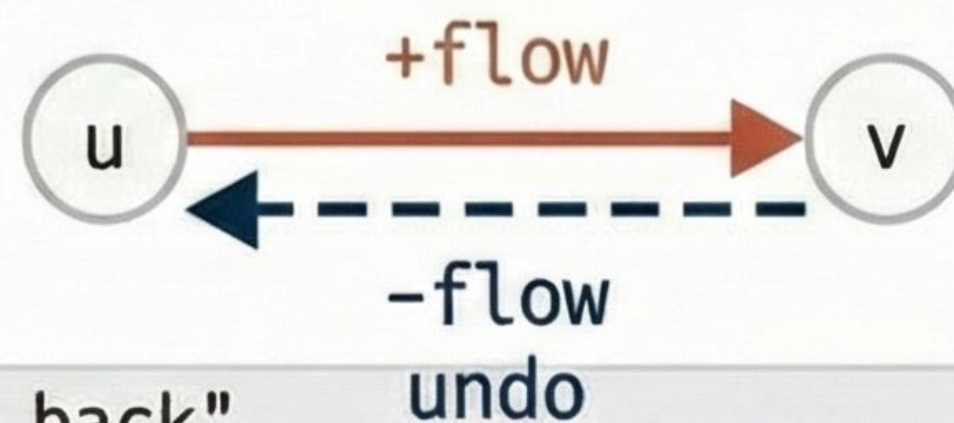
```
def add_edge(self, u, v, cap):  
    # Add the forward edge (u -> v) with its capacity  
    # 添加正向边 (u -> v) 及其容量  
    self.graph[u].append(v)  
    self.capacity[u][v] = cap
```

```
    # Add the backward edge (v -> u) for "sending flow back"  
    # 添加反向边 (v -> u) 用于“回退流量”  
    self.graph[v].append(u)  
    # self.capacity[v][u] is initially 0  
    # self.capacity[v][u] 初始为 0
```

Why? 为什么？

The backward edge allows the algorithm to 'undo' a flow decision if a better path is found later. This is key to how max-flow algorithms work.

反向边允许算法在之后找到更好的路径时“撤销”一个流量分配。这是最大流算法工作的关键。



Step 3 - BFS (Breadth-First Search)

第三步 - BFS (广度优先搜索)

BFS finds the shortest path from Source to Sink with available capacity.

BFS 寻找从源点到汇点且有剩余容量的最短路径。

Initialization

初始化

The search requires a `queue` for exploration and a `visited` array to prevent cycles.

搜索需要一个 `queue` (队列) 来探索, 以及一个 `visited` (已访问) 数组来防止循环。

```
# Part of the MaxFlowSolver Class
def bfs(self, parent):
    visited = [False] * self.num_nodes
    queue = deque([self.source])
    visited[self.source] = True
    parent[self.source] = -1
```

Step 3 - BFS Loop

第三步 - BFS 循环

```
# Inside the bfs function
while queue:
    u = queue.popleft()
    for v in self.graph[u]:
        # If not visited AND capacity > 0
        # 如果未访问 且 容量 > 0
        if not visited[v] and self.capacity[u][v] > 0:
            queue.append(v)
            visited[v] = True
            parent[v] = u
            if v == self.sink:
                return True
    return False
```



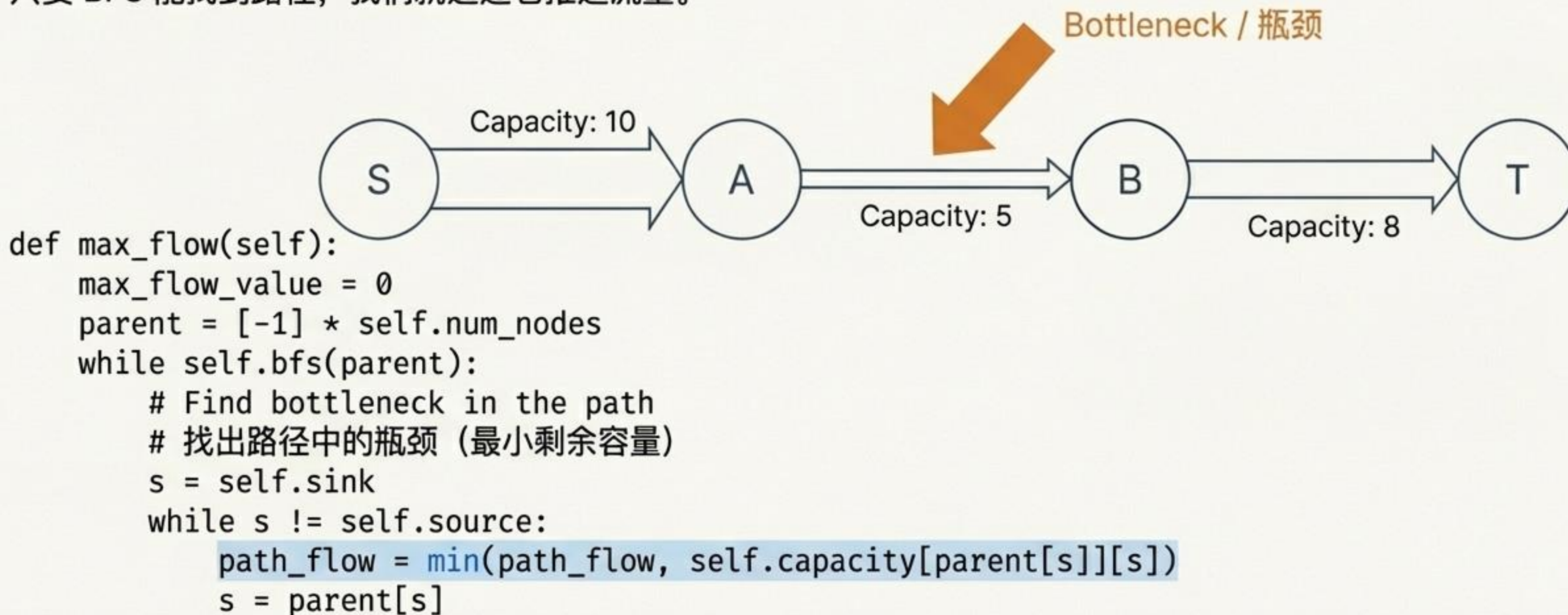
The search advances only along edges with remaining capacity to unvisited nodes.

搜索只沿着有剩余容量且通向未访问节点的边前进。

Step 4 - Max Flow Logic

第四步 - 最大流逻辑

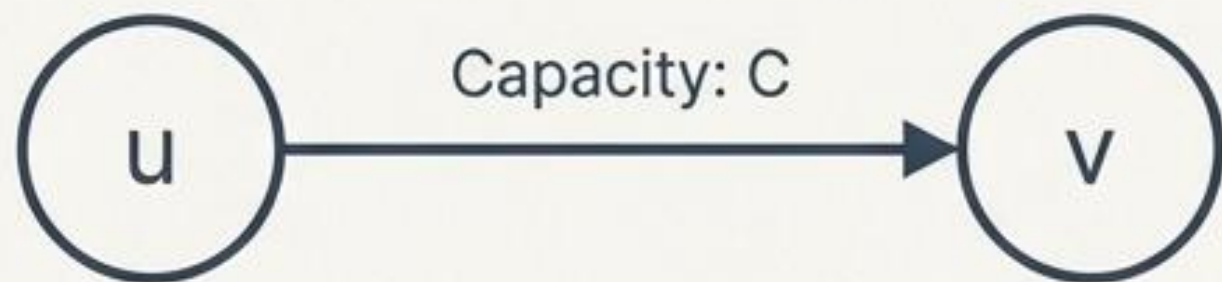
While BFS finds a path, we push flow through it.
只要 BFS 能找到路径，我们就通过它推送流量。



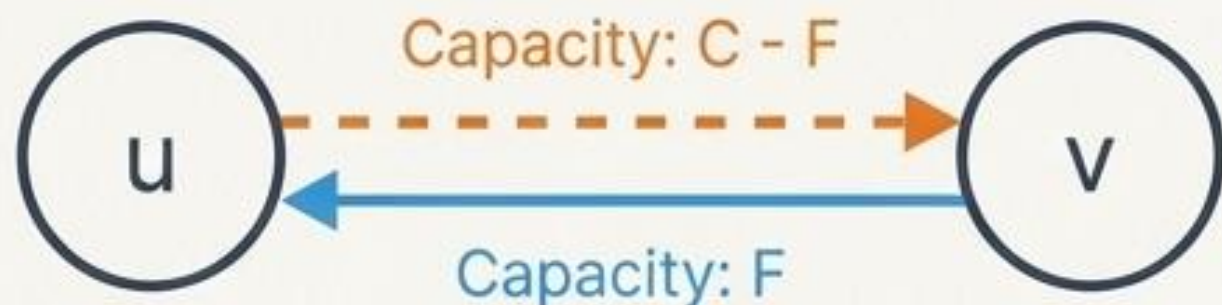
Step 4 - Updating the Residual Graph

第四步 - 更新残差网络

Before / 更新前



After (pushing flow 'F') / 更新后 (推送流量 'F')



```
# Update capacities
```

```
# 更新容量
```

```
max_flow_value += path_flow
```

```
v = self.sink
```

```
while v != self.source:
```

```
    u = parent[v]
```

```
    self.capacity[u][v] -= path_flow # Decrease forward
```

```
    self.capacity[v][u] += path_flow # Increase backward
```

```
    v = parent[v]
```

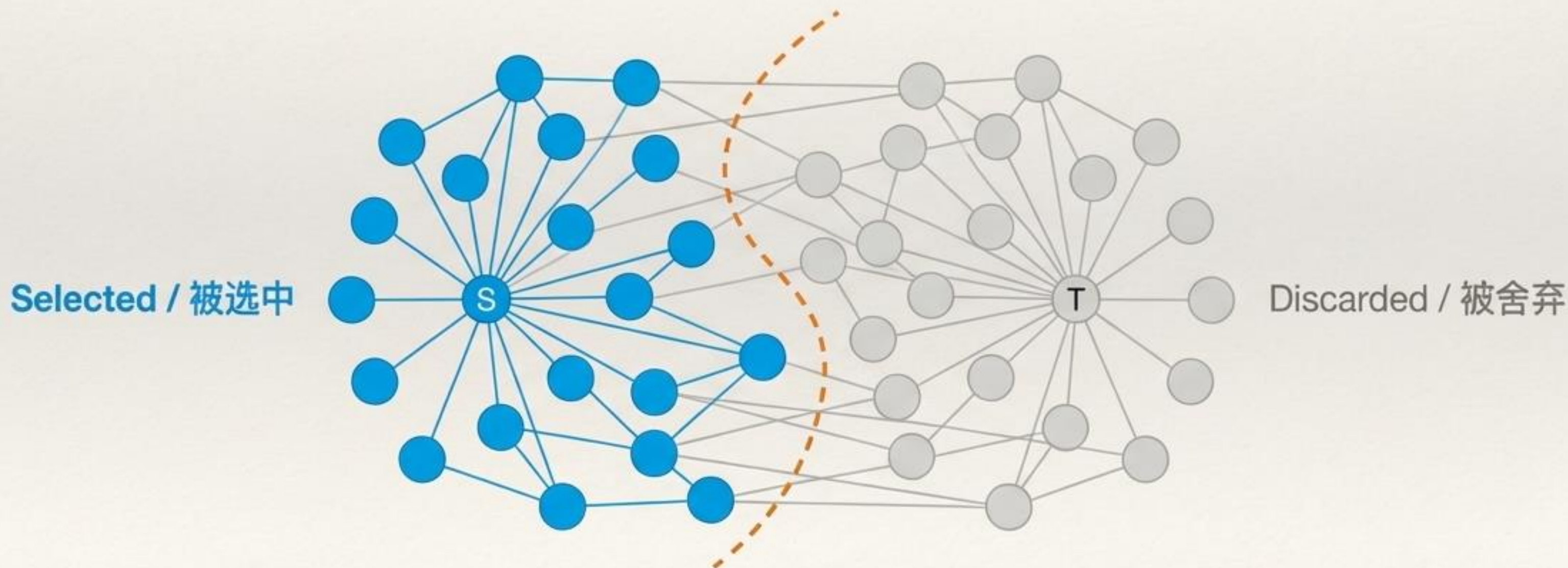
```
return max_flow_value
```

Step 5 - Finding Selected Items

第五步 - 找出被选中的项目

How do we know which experiments were chosen?
我们如何知道选择了哪些实验？

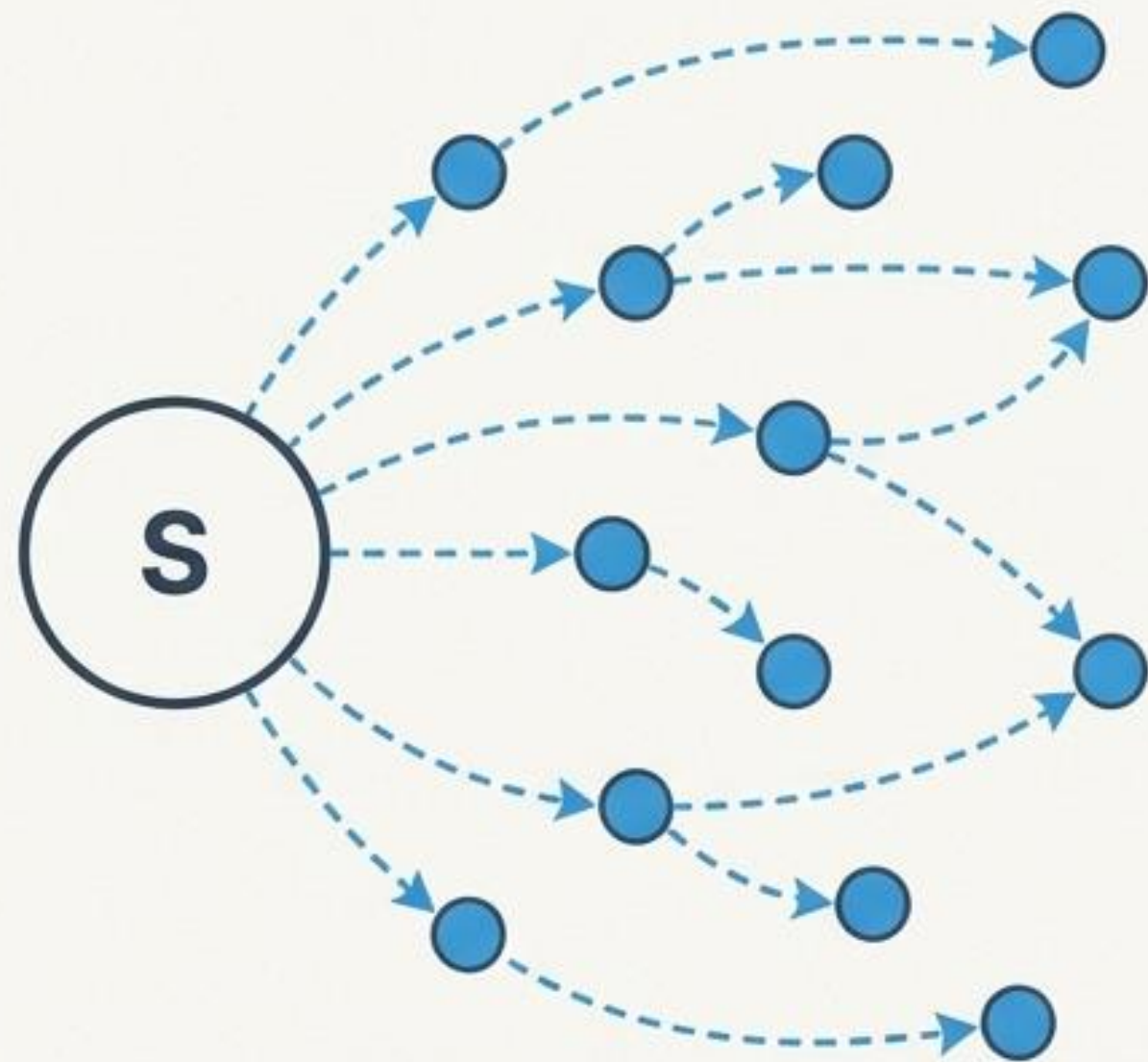
All nodes **reachable** from Source in the residual graph are 'Selected'.
在残余网络中，所有从源点 **可达** 的节点即为“被选中”的。



Step 5 - Implementation

第五步 - 代码实现

```
def get_selected_set(self):  
    visited = [False] * self.num_nodes  
    queue = deque([self.source])  
    visited[self.source] = True  
  
    # Simple BFS from Source  
    # 从源点开始简单的 BFS  
    while queue:  
        u = queue.popleft()  
        for v in self.graph[u]:  
            if not visited[v] and self.capacity[u][v] > 0:  
                visited[v] = True  
                queue.append(v)  
    return visited
```



A final search from the Source identifies every reachable node, defining the 'Selected' set.
从源点开始的最后一次搜索将识别所有可达节点，从而定义“被选中”的集合。

Step 6 - Reading Input (Main Function)

第六步 - 读取输入（主函数）

Now we parse the input file `input.txt`.
现在我们要解析输入文件 `input.txt`。



Logical Flow: 逻辑流程:

1. Read `m` (experiments) and `n` (instruments).
读取 `m` (实验数) 和 `n` (仪器数)。
2. Read `m` lines, each containing data for one experiment.
读取 `m` 行，每行包含一个实验的数据。
3. Read the final line for all instrument costs.
读取最后一行所有仪器的成本。

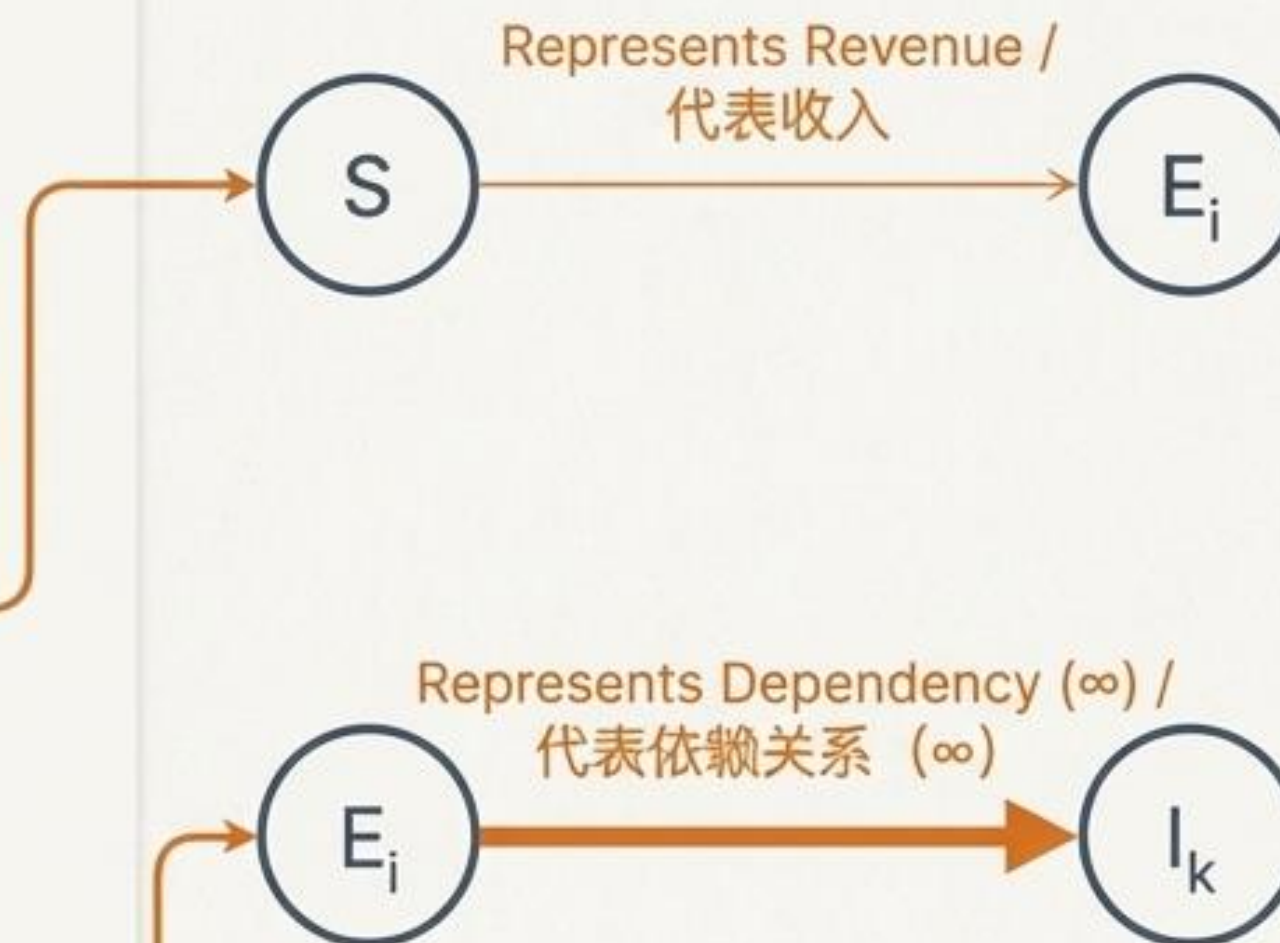
Building the Experiment Edges

构建实验相关的边

```
# Iterate experiments 1 to m
# 遍历实验 1 到 m
for i in range(1, m + 1):
    # Parse: Revenue, Instr_1, Instr_2...
    parts = list(map(int, lines[i].split()))
    revenue = parts[0]

    # Edge: Source -> Experiment (Cap = Revenue)
    # 边: 源点 -> 实验 (容量 = 收入)
    solver.add_edge(source, i, revenue)

    # Edges: Experiment -> Instruments (Cap = Inf)
    # 边: 实验 -> 仪器 (容量 = 无穷大)
    for instr_id in parts[1:]:
        solver.add_edge(i, m + instr_id, float('inf'))
```



Building the Instrument Edges

构建仪器相关的边

```
costs = list(map(int, lines[m + 1].split()))
for i in range(n):
    cost = costs[i]
    # Edge: Instrument -> Sink (Cap = Cost)
    # 边: 仪器 -> 汇点 (容量 = 费用)
    # Node mapping: Instrument i+1 is at index m+i+1
    solver.add_edge(m + i + 1, sink, cost)
```



Node Index = `m` + Instrument ID
节点索引 = `m` + 仪器 ID

Calculating the Final Result

计算最终结果



1. Compute Minimum Loss

1. 计算最小损失

```
# 1. Compute Min Loss (Max Flow)  
min_loss = solver.max_flow()
```



2. Compute Maximum Profit

2. 计算最大利润

```
# 2. Compute Max Profit  
max_profit = total_revenue - min_loss
```



3. Identify Selected Items

3. 获取被选中的项目

```
# 3. Get Selected Items  
visited_nodes = solver.get_selected_set()
```

Formatting the Output

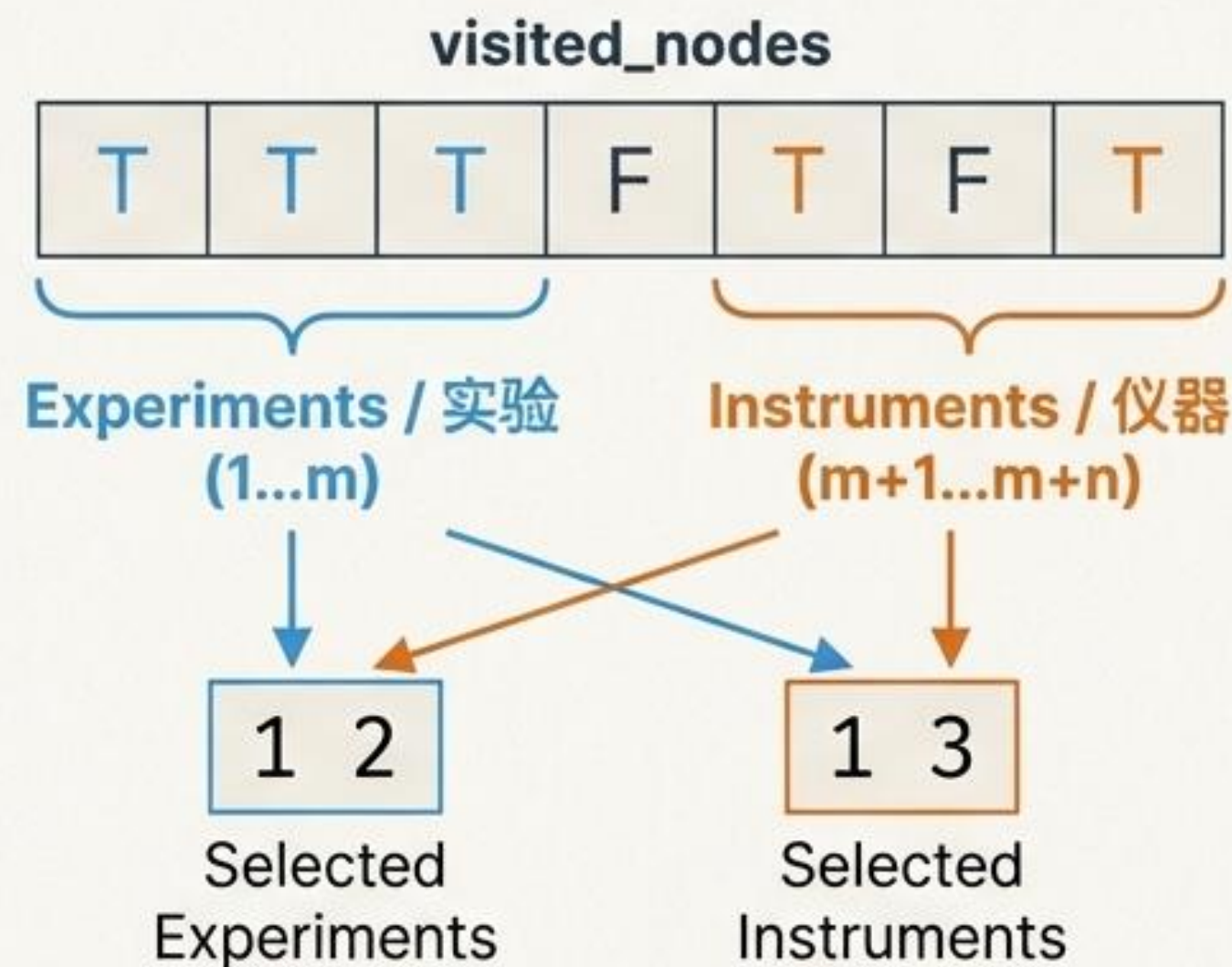
格式化输出

Finally, we print the selected Experiment IDs and Instrument IDs.
最后，我们打印选中的实验 ID 和仪器 ID。

```
// Loop through experiment indices (1 to m)
// 遍历实验索引 (1 至 m)
if visited_nodes[i] is true:
    add i to selected_experiments

// Loop through instrument indices (m+1 to m+n)
// 遍历仪器索引 (m+1 至 m+n)
if visited_nodes[j] is true:
    add instrument_id to selected_instruments

// print results
// 打印结果
```



Time Complexity

时间复杂度

Edmonds-Karp Complexity: $O(V \cdot E^2)$

Edmonds-Karp 时间复杂度: $O(V \cdot E^2)$

For larger data, we use **Dinic's Algorithm** ($O(V^2 \cdot E)$).
对于更大的数据, 我们使用 **Dinic 算法** ($O(V^2 \cdot E)$)。

Note: In our graph... 注意: 在我们的图中...

The number of vertices $V = m + n + 2$. 顶点数 $V = m + n + 2$ 。

The number of edges E is approximately $m + n + (\text{total number of dependencies})$.
边数 E 约等于 $m + n + (\text{总依赖关系数})$ 。

Visualizing the Result (Example)

结果可视化（示例）

The Max Flow was 18.
最大流是 18。



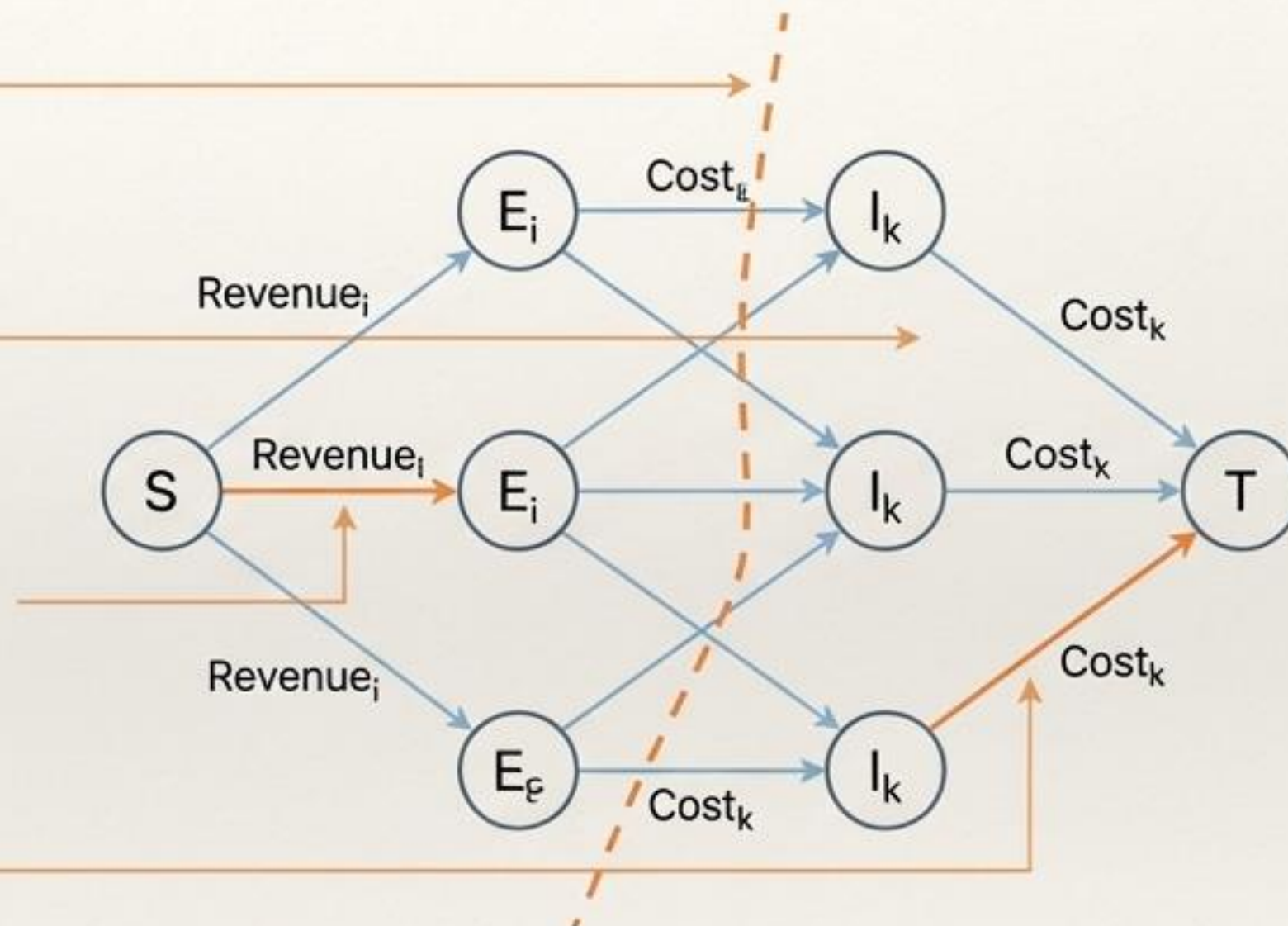
$$\text{Total Potential (35)} - \text{Loss (18)} = \text{Profit (17)}$$

总潜力 (35) - 损失 (18) = 利润 (17)

Why This Works (Summary)

原理回顾 (总结)

- Source side = Selected
源点一侧 = 被选中
- Sink side = Discarded
汇点一侧 = 被舍弃
- Cut Revenue Edge = Lost Opportunity
割断收入边 = 失去机会
- Cut Cost Edge = Paid Cost
割断成本边 = 支付成本



Conclusion

总结

The Space Flight Planning problem is a classic example of **Project Selection (or Maximum Weight Closure).**

太空飞行计划问题是 **项目选择**（或最大权闭合子图）的经典案例。

We successfully mapped a Profit Maximization problem to a Minimum Cut Graph problem.

我们成功地将利润最大化问题映射为最小割图论问题。